What should Ubisoft's next game be?

A market study of the **55 690 games** listed on Steam, run for Ubisoft's studio leadership.

Jedha *Full Stack Data Scientist* — **Block 2, Big Data Project**. PySpark on Databricks.

Ubisoft wants to launch a new game and asked for a global reading of the Steam marketplace before the concept is locked. The brief lists a dozen questions on three levels — the market as a whole, genres, and platforms. This notebook takes the three levels in that order and answers the twelve questions, nothing more.

Section 6 gathers what those answers imply for the product brief, and separates what is measured from what is inferred. Section 7 says what this dataset cannot decide at all.

Where each question is answered

#	The brief asks	Answered in
	Macro	
1	Which publisher has released the most games on Steam?	3.1
2	What are the best rated games?	**3.6**
3	Are there years with more releases? More or fewer during Covid?	3.2
4	How are the prices distributed? Are there many games with a discount?	3.3
5	What are the most represented languages?	3.4
6	Are there many games prohibited for children under 16/18?	3.5
	Genres	
7	What are the most represented genres?	4.1
8	Are there any genres with a better positive/negative review ratio?	4.2
9	Do some publishers have favourite genres?	4.3
10	What are the most lucrative genres?	4.4
	Platforms	
11	Are most games available on Windows/Mac/Linux?	5.1
12	Do certain genres tend to be available on certain platforms?	5.2

Question 2 closes the macro level instead of coming second: the best-rated list produces a benchmark to hit rather than an answer the other macro questions build on.

One section goes beyond the list, because the brief's stated goal — *what factors affect the popularity or sales of a video game* — needs it: **4.5** tests whether any genre is actually emerging.

The data

`steam_game_output.json` — a 61 MB JSON array, one object per game, `{"id": ..., "data": {...}}`, served from `s3://full-stack-bigdata-datasets/Big_Data/Project_Steam/`. It is a **SteamSpy snapshot** and the newest release in it is **11 November 2022**, so every count for 2022 is partial and no game released after that date exists here.

Three of the 22 fields are nested — `tags` (a tag → number-of-votes object), `platforms` (three booleans) and `categories` (a list) — which is what makes the file semi-structured and what `explode()` and `getField()` are for.

0. Environment

The notebook is written for **Databricks**, and falls back to a local Spark session so the code can be re-run outside a workspace. Everything below this cell is identical in both cases, including the `display()` calls that drive Databricks' visualisation tool.

Two things differ per environment and are hidden behind the same names: `display()`, and `materialise()` — serverless compute has no `cache()`, so a frame that many later cells re-read is written to a Delta table there, and simply cached locally.

```
import os

IS_DATABRICKS = "DATABRICKS_RUNTIME_VERSION" in os.environ
SOURCE_URL = ("https://full-stack-bigdata-datasets.s3.amazonaws.com"
              "/Big_Data/Project_Steam/steam_game_output.json")

if IS_DATABRICKS:
    # Unity Catalog volume: the file is fetched once, then read from storage.
    spark.sql("CREATE VOLUME IF NOT EXISTS workspace.default.steam")
    DATA_PATH = "/Volumes/workspace/default/steam/steam_game_output.json"
    if not os.path.exists(DATA_PATH):
        import urllib.request
        urllib.request.urlretrieve(SOURCE_URL, DATA_PATH)

    def materialise(df, name):
        # No cache() on serverless. A Delta table costs one write and turns the
        # non-splittable JSON into columnar storage for every read that follows.
        table = f"workspace.default.{name}"
        df.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable(table)
        return spark.table(table)
else:
    # Local run: build the session by hand and emulate Databricks' display().
    from pyspark.sql import SparkSession, DataFrame
    from IPython.display import display as _ipython_display

    os.environ.setdefault("SPARK_LOCAL_IP", "127.0.0.1")
    spark = (SparkSession.builder.appName("steam")
             .master("local[*]")
             .config("spark.driver.memory", "10g")
             .config("spark.sql.session.timeZone", "UTC")
             .config("spark.ui.showConsoleProgress", "false")
             .getOrCreate())
    spark.sparkContext.setLogLevel("ERROR")

    def display(x, n=1000):
        if isinstance(x, DataFrame):
            # toPandas() widens a nullable int column to float; Int64 keeps it an
            # integer with <NA>, which is what Databricks' own display() shows.
            ints = [f.name for f in x.schema.fields
                    if isinstance(f.dataType, (IntegerType, LongType))]
            x = x.limit(n).toPandas().astype({c: "Int64" for c in ints})
        _ipython_display(x)

    def materialise(df, name):
        return df.cache()

    DATA_PATH = "data/steam_game_output.json"

from pyspark.sql import functions as F, Window
from pyspark.sql.types import (StructType, StructField, StringType, IntegerType,
                               LongType, BooleanType, ArrayType, MapType)

print("Spark", spark.version, "| Databricks" if IS_DATABRICKS else "| local", "|", DATA_PATH)
```

```
Spark 4.2.0 | Databricks | /Volumes/workspace/default/steam/steam_game_output.json
```

1. Reading a semi-structured file

The whole array sits on **one line**, so `multiLine` has to be on: without it Spark splits the file on newlines and finds a single unparseable record.

1.1 Why schema inference is not enough

Let Spark infer the schema and `tags` — an object whose *keys are the tag names* — becomes a struct with one column per tag seen anywhere in the file. Inference also has to scan the 61 MB twice, and it silently picks a type for `required_age`, a field that holds both numbers and strings.

```
inferred = spark.read.option("multiLine", True).json(DATA_PATH)

tags_field = inferred.schema["data"].dataType["tags"].dataType
print("columns Spark invented for `tags`:", len(tags_field.fields))
print("first five:", [f.name for f in tags_field.fields[:5]])
```

```
inferred:  pyspark.sql.connect.dataframe.DataFrame = [data: struct, id: string]
columns Spark invented for `tags`: 441
first five: ['1980s', "1990's", '2.5D', '2D', '2D Fighter']
```

1.2 An explicit schema instead

Declaring `tags` as `MAP<STRING, BIGINT>` turns those 441 phantom columns into one map column that `explode()` can open into (tag, votes) rows. `required_age` is declared `STRING` and parsed later, where the parsing rule is visible.

```
DATA_SCHEMA = StructType([
    StructField("appid", LongType()),
    StructField("name", StringType()),
    StructField("short_description", StringType()),
    StructField("developer", StringType()),
    StructField("publisher", StringType()),
    StructField("genre", StringType()),                      # comma-separated
    StructField("tags", MapType(StringType(), LongType())),  # tag -> community votes
    StructField("type", StringType()),
    StructField("categories", ArrayType(StringType())),
    StructField("owners", StringType()),                     # bucketed range, e.g. "0 .. 20,000"
    StructField("positive", LongType()),
    StructField("negative", LongType()),
    StructField("price", StringType()),                      # US cents, as a string
    StructField("initialprice", StringType()),
    StructField("discount", StringType()),                   # percent, as a string
    StructField("ccu", LongType()),                          # peak concurrent users
    StructField("languages", StringType()),                  # comma-separated
    StructField("platforms", StructType([
        StructField("windows", BooleanType()),
        StructField("mac", BooleanType()),
        StructField("linux", BooleanType()),
    ])),
    StructField("release_date", StringType()),
    StructField("required_age", StringType()),
    StructField("website", StringType()),
    StructField("header_image", StringType()),
])

SCHEMA = StructType([
    StructField("id", StringType()),
    StructField("data", DATA_SCHEMA),
])

raw = (spark.read.schema(SCHEMA).option("multiLine", True).json(DATA_PATH)
       .select("id", "data.*"))          # flatten the nested `data` struct

print(f"{raw.count():,} games x {len(raw.columns)} columns")
raw.printSchema()
```

```
raw:  pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 21 more fields]
```

```
55,691 games x 23 columns
root
 |-- id: string (nullable = true)
 |-- appid: long (nullable = true)
 |-- name: string (nullable = true)
 |-- short_description: string (nullable = true)
 |-- developer: string (nullable = true)
 |-- publisher: string (nullable = true)
 |-- genre: string (nullable = true)
 |-- tags: map (nullable = true)
 |    |-- key: string
 |    |-- value: long (valueContainsNull = true)
 |-- type: string (nullable = true)
 |-- categories: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- owners: string (nullable = true)
 |-- positive: long (nullable = true)
 |-- negative: long (nullable = true)
 |-- price: string (nullable = true)
 |-- initialprice: string (nullable = true)
 |-- discount: string (nullable = true)
```

1.3 The map opens

`explode()` on the `tags` map is what gives every later tag question a row-per-tag table.

```
display(
    raw.filter(F.col("name") == "Counter-Strike")
       .select("name", F.explode("tags").alias("tag", "votes"))
       .orderBy(F.desc("votes")).limit(8)
)
```

Table

	name	tag	votes
1	Counter-Strike	Action	5426
2	Counter-Strike	FPS	4831
3	Counter-Strike	Multiplayer	3392
4	Counter-Strike	Shooter	3353
5	Counter-Strike	Classic	2784
6	Counter-Strike	Team-Based	1864
7	Counter-Strike	First-Person	1707
8	Counter-Strike	Competitive	1607

8 rows

2. Cleaning

Nine rules. Each one is stated with the number of rows it touches, so nothing is silently dropped or rewritten.

2.1 Keys and scope

`id` duplicates `appid`, and every `appid` is unique — there is nothing to deduplicate. One row is not a game.

```
print("rows where id != appid:", raw.filter(F.col("id") != F.col("appid").cast("string")).count())
print("distinct appid:", raw.select("appid").distinct().count(), "of", raw.count(), "rows")
display(raw.groupBy("type").count())
```

```
rows where id != appid: 0
distinct appid: 55691 of 55691 rows
```

Table

2.2 Money

`price`, `initialprice` and `discount` are strings. Prices are US cents, so a `999` is $9.99. `price` is the current price and `initialprice` the list price: the two differ on exactly the 2 518 rows that carry a discount, so the field is internally consistent and `initialprice` is the one to use for anything about a game's positioning.

```
games = (raw.filter(F.col("type") == "game")
         .withColumn("price_usd", F.col("price").cast("int") / 100)
         .withColumn("initial_price_usd", F.col("initialprice").cast("int") / 100)
         .withColumn("discount_pct", F.col("discount").cast("int"))
         .withColumn("is_free", F.col("price").cast("int") == 0))

display(games.select(
    F.round(F.min("price_usd"), 2).alias("min_price"),
    F.round(F.max("price_usd"), 2).alias("max_price"),
    F.sum(F.col("is_free").cast("int")).alias("free_games"),
    F.sum((F.col("discount_pct") > 0).cast("int")).alias("discounted"),
    F.sum((F.col("price") != F.col("initialprice")).cast("int")).alias("price_below_list"),
))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 25 more fields]

Table

2.3 Release dates

Four shapes in one column: `2020/06/18`, `2020/06/8` (one-digit day), `2015/09` (no day at all) and empty. Matching each shape before parsing keeps the 99 empty strings as an explicit `NULL` instead of a silent failure.

```
games = (games
    .withColumn("release_date_parsed",
        F.when(F.col("release_date").rlike(r"^\d{4}/\d{1,2}/\d{1,2}$"),
               F.to_date("release_date", "yyyy/M/d"))
         .when(F.col("release_date").rlike(r"^\d{4}/\d{1,2}$"),
               F.to_date("release_date", "yyyy/M")))
    .withColumn("release_year", F.year("release_date_parsed"))
    .withColumn("release_month", F.month("release_date_parsed")))

display(games.select(
    F.min("release_date_parsed").alias("first_release"),
    F.max("release_date_parsed").alias("last_release"),
    F.sum(F.col("release_date_parsed").isNull().cast("int")).alias("unparseable"),
))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 28 more fields]

Table

	first_release	last_release	unparseable
1	1997-06-30	2022-11-11	99

1 row

2.4 Comma-separated lists

`genre` and `languages` pack several values into one string. Splitting on the comma is not quite enough: four rows carry a parenthetical note (`English (full audio)`), two a stray semicolon, and one lists English twice. Stripping the noise, trimming and deduplicating gives clean arrays — and note that `Spanish - Spain` and `Design & Illustration` mean the separator can only ever be the comma.

```
def split_list(column):
    """Comma-separated string -> trimmed, deduplicated array, parenthetical notes removed."""
    parts = F.split(F.regexp_replace(F.col(column), r"\([^)]*\)|[;*]", ""), ",")
    return F.array_distinct(F.array_remove(F.transform(parts, lambda x: F.trim(x)), ""))

games = (games
    .withColumn("genres", split_list("genre"))
    .withColumn("languages_list", split_list("languages"))
    .withColumn("n_genres", F.size("genres"))
    .withColumn("n_languages", F.size("languages_list")))

display(games.filter(F.col("languages").contains("(")).select("name", "languages", "languages_list"))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 32 more fields]

Table

2.5 Publisher names

Ubisoft appears under five spellings, two of which differ only by a trademark sign or four trailing tabs. Trimming whitespace and stripping `®` / `™` rewrites 271 rows and empties 134 more that held nothing but whitespace — 405 changes in all. The report in 2.10 therefore compares with `eqNullSafe` and not `!=`, or SQL's `NULL != 'x' -> NULL` would silently drop those 134. It does **not** merge `Ubisoft` with `Ubisoft Entertainment`, and it should not — deciding that two different company names are the same firm is a judgement call, not a cleaning rule. The publisher counts in section 3.1 are therefore a floor, not an exact figure.

The field also holds co-publisher lists (`Team17, NEXT Studios`), but 3 215 rows contain a comma and most of them are `Ltd.`-style suffixes, so splitting on it would create more noise than it removes. The string is kept whole.

```
def normalise_name(column):
    cleaned = F.trim(F.regexp_replace(F.regexp_replace(F.col(column), r"[®™]", ""), r"\s+", " "))
    return F.when(cleaned != "", cleaned)

games = games.withColumn("publisher_clean", normalise_name("publisher"))

print("distinct publisher strings:", games.select("publisher").distinct().count(),
      "-> after normalisation:", games.select("publisher_clean").distinct().count())

display(games.filter(F.col("publisher").rlike("^Ubisoft"))
             .groupBy("publisher", "publisher_clean").count().orderBy(F.desc("count")))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 33 more fields]

```
distinct publisher strings: 29966 -> after normalisation: 29825
```

Table

2.6 Owners

SteamSpy does not publish a sales figure, it publishes a bracket: `"10,000,000 .. 20,000,000"`. Two regexes give the bounds, and the midpoint stands in for the value. **68% of the catalogue sits in the bottom bracket**, so that midpoint is `10,000` for two games out of three and is useless as a ranking on its own. Nothing here ranks on it directly: it enters only as one factor of `owners_x_price` (2.9), and section 7 takes that apart.

```
owners_digits = F.regexp_replace(F.col("owners"), ",", "")
games = (games
    .withColumn("owners_min", F.regexp_extract(owners_digits, r"^(\d+)", 1).cast("long"))
    .withColumn("owners_max", F.regexp_extract(owners_digits, r"\.\.\s*(\d+)$", 1).cast("long")))
games = games.withColumn("owners_mid", (F.col("owners_min") + F.col("owners_max")) / 2)

display(games.groupBy("owners", "owners_min", "owners_max").count().orderBy("owners_min"))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 36 more fields]

Table

2.7 Required age

The field mixes integers and strings, and its odd values are `"MA 15+"`, `"21+"`, `"7+"`, `"35"` and `"180"` (four times). Pulling the first number out handles the `+` suffixes; anything outside 0-21 is not an age rating and becomes `NULL` — five rows.

That fixes the parsing but not the field: **it is 0 for 98.8% of the catalogue**, because Steam gates mature content through its own content descriptors rather than this legacy attribute. Section 3.5 answers the brief's question about age-restricted games from the community tags instead.

```
age = F.regexp_extract(F.col("required_age"), r"(\d+)", 1).cast("int")
games = games.withColumn("age_rating", F.when(age.between(0, 21), age))

display(games.groupBy("required_age", "age_rating").count().orderBy(F.desc("count")).limit(25))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 37 more fields]

Table

2.8 Reviews

`positive` and `negative` are review counts, not scores. The raw share of positive reviews is unusable as a ranking on its own: **8 634 games sit at exactly 100%**, most of them on a handful of reviews. The Wilson 95% lower bound answers the question actually being asked — *how good is this game, given how little we know about it* — by pulling small samples towards the middle.

```
games = (games
    .withColumn("reviews", F.col("positive") + F.col("negative"))
    .withColumn("positive_ratio",
                F.when(F.col("positive") + F.col("negative") > 0,
                       F.col("positive") / (F.col("positive") + F.col("negative")))))

z, n, p = F.lit(1.96), F.col("reviews"), F.col("positive_ratio")
wilson_lower_bound = (p + z * z / (2 * n) - z * F.sqrt((p * (1 - p) + z * z / (4 * n)) / n)) / (1 + z * z / n)
games = games.withColumn("wilson_score", F.when(n > 0, wilson_lower_bound))

display(games.filter(F.col("positive_ratio") == 1)
             .select("name", "positive", "negative", "positive_ratio", "wilson_score")
             .orderBy("reviews").limit(5))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 40 more fields]

Table

	name	positive	negative	positive_ratio	wilson_score
1	M.C.I. Escapes	1	0	1	0.206543291473892...
2	Kill Tiger	1	0	1	0.206543291473892...
3	Twisting Mower	1	0	1	0.206543291473892...
4	Zero spring episode 3	1	0	1	0.206543291473892...
5	CrossTrix	1	0	1	0.206543291473892...

5 rows

2.9 Platforms, and a value proxy

The `platforms` struct becomes three flags and a count. Revenue is not in the dataset, and nothing here stands in for it: `owners` counts copies *held*, not copies bought — bundles, gift keys and free weekends all land in it — and `initialprice` is the list price, not a price anyone paid. And `owners` is not even a count: SteamSpy publishes a bracket, so what enters the product is its midpoint (2.6). Their product is named for what it is, **`owners_x_price` : the list-price value of the owners bracket's midpoint**. Section 4.4 uses it as a sort key; section 7 takes it apart.

```
games = (games
    .withColumn("windows", F.col("platforms.windows"))
    .withColumn("mac", F.col("platforms.mac"))
    .withColumn("linux", F.col("platforms.linux"))
    .withColumn("n_platforms", F.col("platforms.windows").cast("int")
                             + F.col("platforms.mac").cast("int")
                             + F.col("platforms.linux").cast("int"))
    .withColumn("owners_x_price", F.col("owners_mid") * F.col("initial_price_usd")))

games = materialise(games, "steam_games")
print(f"{games.count():,} games ready, {len(games.columns)} columns")
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 45 more fields]

```
55,690 games ready, 47 columns
```

2.10 Cleaning report

```
report = spark.createDataFrame([
    ("2.1   drop type != 'game'",           games.count(), "rows dropped",
     raw.count() - games.count()),
    ("2.2   price strings -> USD",          games.count(), "non-numeric source",
     games.filter(~F.col("price").rlike(r"^-?\d+$")
                | ~F.col("initialprice").rlike(r"^-?\d+$")
                | ~F.col("discount").rlike(r"^-?\d+$")).count()),
    ("2.3   release_date -> date",          games.count(), "source empty -> NULL",
     games.filter(F.col("release_date_parsed").isNull()).count()),
    ("2.4a  genre -> array",                games.count(), "source empty -> []",
     games.filter(F.col("n_genres") == 0).count()),
    ("2.4b  languages -> array",            games.count(), "source empty -> []",
     games.filter(F.col("n_languages") == 0).count()),
    ("2.5   publisher name normalised",     games.count(), "values rewritten",
     games.filter(~F.col("publisher").eqNullSafe(F.col("publisher_clean"))).count()),
    ("2.6   owners bracket -> bounds",      games.count(), "unparsed -> NULL",
     games.filter(F.col("owners_min").isNull()).count()),
    ("2.7   required_age -> 0-21 or NULL",  games.count(), "out of range -> NULL",
     games.filter(F.col("age_rating").isNull()).count()),
    ("2.8   reviews -> Wilson score",       games.count(), "no reviews -> NULL",
     games.filter(F.col("wilson_score").isNull()).count()),
], ["rule", "rows_in", "measured", "n"])

display(report)
```

report: pyspark.sql.connect.dataframe.DataFrame = [rule: string, rows_in: long ... 2 more fields]

Table

3. The market

3.1 Who publishes on Steam

Chart: bar, `publisher_clean` *x* `games`*.*

```
by_publisher = (games.groupBy("publisher_clean")
    .agg(F.count("*").alias("games"))
    .filter(F.col("publisher_clean").isNotNull()))

display(by_publisher.orderBy(F.desc("games")).limit(20))
```

by_publisher: pyspark.sql.connect.dataframe.DataFrame = [publisher_clean: string, games: long]

Table **Visualization 1**

Big Fish Games has released the most games — 423, casual hidden-object titles — ahead of 8floor (202) and SEGA (165). Ubisoft is tenth with 128.

That ranking says less than the shape behind it. The 55 690 games spread over 29 824 named publishers — 134 games carry no publisher at all, and appear in no ranking here — and the concentration is the finding:

```
buckets = (by_publisher
    .withColumn("size", F.when(F.col("games") == 1, "1 game")
                        .when(F.col("games") <= 5, "2-5 games")
                        .when(F.col("games") <= 20, "6-20 games")
                        .otherwise("21+ games"))
    .groupBy("size").agg(F.count("*").alias("publishers"), F.sum("games").alias("games")))

# by_publisher drops the games with no publisher, so the buckets cover 55 556 of the 55 690.
# The residual row keeps the numerator on the same population as the denominator below.
orphans = (games.filter(F.col("publisher_clean").isNull())
    .groupBy(F.lit("no publisher").alias("size"))
    .agg(F.lit(0).cast("long").alias("publishers"), F.count("*").alias("games")))

concentration = (buckets.unionByName(orphans)
    .withColumn("pct_of_catalogue", F.round(100 * F.col("games") / games.count(), 1)))

# the total row holds the two catalogue-wide figures the text above quotes
total = concentration.agg(
    F.lit("all").alias("size"),
    F.sum("publishers").alias("publishers"),
    F.sum("games").alias("games"),
    F.round(100 * F.sum("games") / games.count(), 1).alias("pct_of_catalogue"))

display(concentration.unionByName(total)
        .orderBy(F.col("size").isin("no publisher", "all"), "publishers"))
```

- buckets: pyspark.sql.connect.dataframe.DataFrame = [size: string, publishers: long ... 1 more field]
- concentration: pyspark.sql.connect.dataframe.DataFrame = [size: string, publishers: long ... 2 more fields]
- orphans: pyspark.sql.connect.dataframe.DataFrame = [size: string, publishers: long ... 1 more field]
- total: pyspark.sql.connect.dataframe.DataFrame = [size: string, publishers: long ... 2 more fields]

Table

41% of the catalogue comes from publishers that have released exactly one game. Steam is not a market of a few big houses — it is a very long tail, and the 195 publishers with 21 releases or more hold 17% of it.

3.2 The release calendar

Chart: bar, `release_year` *x* `games`*.*

```
per_year = (games.filter(F.col("release_year").isNotNull())
                 .groupBy("release_year").agg(F.count("*").alias("games"))
                 .orderBy("release_year"))
display(per_year)
```

per_year: pyspark.sql.connect.dataframe.DataFrame = [release_year: integer, games: long]

Table **release_year_games**

Two things are stacked in this column, and they separate around 2013. Before it the table counts a curated storefront rather than a market: Steam only opened to third-party publishers in 2005 (Rag Doll Kung Fu and Darwinia, appid 1002 and 1500), and Valve picked by hand what went on sale — 61 games in 2006, still only 471 in 2013. Greenlight opens the gate at the end of 2012 and Steam Direct replaces it in 2017, and the two jumps in the table sit on that calendar: **471 to 1 557 in 2014**, then **4 185 to 6 017 in 2017**. Those two dates are Steam's history and not a measurement from this file, but they are why the early years cannot be read as an industry releasing fewer games. Those years are also a survivors' list — a November 2022 snapshot holds only what was still on sale (section 7).

Covid neither slowed releases nor set them off. 7 678 in 2018, 6 968 in 2019, then 8 305 in 2020 and 8 823 in 2021: the only dip is the year *before* the pandemic, and 2020-2021 extends a plateau that starts in 2018 rather than breaking it. 2022 reads 7 455, but `last_release` in 2.3 is 11 November 2022 — that year is ten and a half months long, and its fall is an artefact.

Inside the year, over the eight complete years from 2014, where the jump above turns the catalogue into a market, to 2021, the last full year the snapshot covers:

Chart: bar, `release_month` *x* `games`*.*

```
display(games.filter(F.col("release_year").between(2014, 2021))
             .groupBy("release_month").agg(F.count("*").alias("games")).orderBy("release_month"))
```

Table **release_month_games**

The calendar is not flat. **October is the busiest month at 4 451 releases and January the quietest at 3 096**, 44% apart, and the shape is a season rather than noise: the six lightest months of the year are exactly January to June, the six heaviest exactly July to December, with a second trough in June. The second half is the run-up to the holiday sales, and it is where the competition for a store slot sits.

The table counts competitors, not buyers — it says how many games a launch shares its month with, and nothing about whether shipping next to them costs or pays.

Decision — release window: the first half of the year, and not the September-December ramp.

3.3 Price

Chart: bar, `band` *x* `games` *.*

```
# band_rank carries the ordering, so the labels can read as prices and nothing else
band_rank = (F.when(F.col("is_free"), 0)
               .when(F.col("price_usd") < 5, 1)
               .when(F.col("price_usd") < 10, 2)
               .when(F.col("price_usd") < 20, 3)
               .when(F.col("price_usd") < 40, 4)
               .otherwise(5))

price_band = (F.when(band_rank == 0, "free")
                .when(band_rank == 1, "under $5")
                .when(band_rank == 2, "$5-10")
                .when(band_rank == 3, "$10-20")
                .when(band_rank == 4, "$20-40")
                .otherwise("$40+"))

display(games.filter(~F.col("is_free")).select(
    F.round(F.min("price_usd"), 2).alias("min_paid"),
    F.round(F.percentile_approx("price_usd", 0.5), 2).alias("median_paid"),
    F.round(F.avg("price_usd"), 2).alias("mean_paid"),
    F.round(F.stddev("price_usd"), 2).alias("stddev_paid"),
    F.round(F.percentile_approx("price_usd", 0.9), 2).alias("p90"),
    F.round(F.percentile_approx("price_usd", 0.99), 2).alias("p99"),
    F.round(F.max("price_usd"), 2).alias("max_paid"),
    F.round(100 * F.avg(((F.col("price").cast("int") % 100) == 99).cast("int")), 1).alias("pct_ends_99")))

display(games.withColumn("rank", band_rank).withColumn("band", price_band)
        .groupBy("rank", "band").agg(
            F.count("*").alias("games"),
            F.round(100 * F.count("*") / games.count(), 1).alias("pct_games"))
        .orderBy("rank").drop("rank"))
```

Table

Table **Visualization 1**

Steam is a cheap store, and it prices in `.99`: **95.6% of paid games end on those two digits**. The mass sits at the bottom — **under $5 alone is 42.2% of the catalogue**, the largest band of the six, and free or under $10 is 78.6% of it. The paid median is $5.99 against a mean of $8.99; that gap, and a standard deviation of $11.30 on that mean, is a thin tail stretching right to a $999 outlier. The 99th percentile is $49.99.

The second half of the question — *are there many games with a discount* — read on the same bands:

Chart: bar, `band` *x* `pct_discounted`*.*

```
display(games.filter(F.col("discount_pct") > 0).select(
    F.count("*").alias("discounted_games"),
    F.round(100 * F.count("*") / games.count(), 1).alias("pct_of_catalogue"),
    F.round(F.avg("discount_pct"), 1).alias("mean_discount"),
    F.percentile_approx("discount_pct", 0.5).alias("median_discount")))

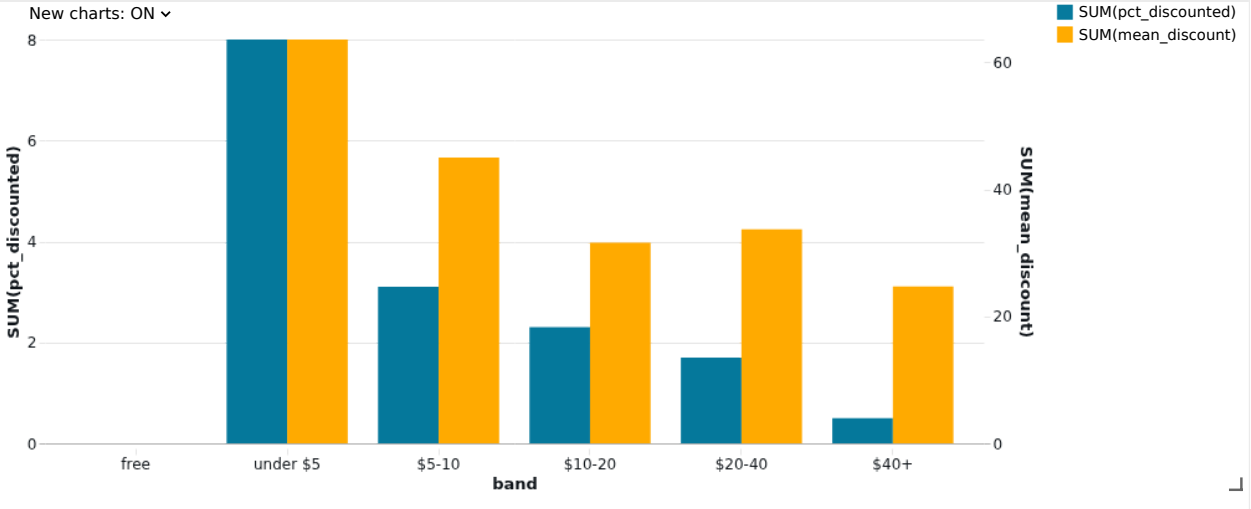
# same bands as above, so the two tables read against each other
on_sale = F.col("discount_pct") > 0
display(games.withColumn("rank", band_rank).withColumn("band", price_band)
        .groupBy("rank", "band").agg(
            F.count("*").alias("games"),
            F.sum(on_sale.cast("int")).alias("discounted"),
            F.round(100 * F.avg(on_sale.cast("int")), 1).alias("pct_discounted"),
            F.round(F.avg(F.when(on_sale, F.col("discount_pct"))), 1).alias("mean_discount"))
        .orderBy("rank").drop("rank"))
```

Table

	discounted_games	pct_of_catalogue	mean_discount	median_discount
1	2518	4.5	57.6	60

1 row

Table **Visualization 1**



6 rows

Not many: 2 518 games are on sale, 4.5% of the catalogue, at a median 60% off. And the discounting is not spread evenly across the store — **1 884 of those 2 518 are games under $5**, the band that is already the largest. Rate and depth both fall with every step up: 8.0% of the under-$5 games are discounted, at a mean 63.6% off, against 0.5% and 24.7% above $40. Cheap games discount often and deep, expensive ones rarely and shallow.

This is a one-day snapshot and Steam's sales are periodic, so it measures the day the file was pulled, not how often a game goes on sale. *Are there many games with a discount* has an answer; *how often does a game go on sale* does not.

3.4 Languages

Chart: bar, `language` *x* `games`*.*

```
by_language = (games.select(F.explode("languages_list").alias("language"))
                    .groupBy("language").agg(F.count("*").alias("games")))

display(by_language
    .withColumn("pct_of_games", F.round(100 * F.col("games") / games.count(), 1))
    .orderBy(F.desc("games")).limit(20))
```

by_language: pyspark.sql.connect.dataframe.DataFrame = [language: string, games: long]

Table **Visualization 1**

English is on 99% of the catalogue and is not a decision — a game that ships in one language ships in English. Below it comes a block of seven: German 25.2%, French 24.1%, Russian 23.2%, Simplified Chinese 23.0%, Spanish 22.0%, Japanese 18.6%, Italian 16.7%. Then the list steps down — Portuguese-Brazil 12.1%, Korean 11.9% — and the twentieth name is already at 3.5%. The widest step in the tail is the one just after Italian, 16.7% to 12.1%.

Which languages is one count. How many of them a game carries is another:

Chart: bar, `languages` *x* `games`*.*

```
language_band = (F.when(F.col("n_languages") <= 1, "1")
                  .when(F.col("n_languages") <= 4, "2-4")
                  .when(F.col("n_languages") <= 9, "5-9")
                  .when(F.col("n_languages") <= 14, "10-14")
                  .when(F.col("n_languages") <= 20, "15-20")
                  .otherwise("21+"))

# min(n_languages) orders the bands without restating the thresholds a second time
display(games.withColumn("languages", language_band)
        .groupBy("languages").agg(
            F.count("*").alias("games"),
            F.round(100 * F.count("*") / games.count(), 1).alias("pct_games"),
            F.min("n_languages").alias("floor"))
        .orderBy("floor").drop("floor"))
```

Table **Visualization 1**

Localisation is the exception on Steam. 29 665 games — 53.3% of the catalogue — ship in a single language, another 23.4% stop at four, and only 5 622, one game in ten, carry more than nine.

So the ranking above is a shortlist, not a description of the average game: English because it is not a choice, then the seven names between 16.7% and 25.2%. What the two tables measure is prevalence and order, not payoff — they say which languages the market translates into, and how rarely it bothers.

Decision — languages: English plus the seven-name tier — German, French, Russian, Simplified Chinese, Spanish, Japanese, Italian. Eight in all, cut where the catalogue's own list steps down.

3.5 Age restriction

Chart: bar, `age_rating` *x* `games`*.*

```
display(games.groupBy("age_rating").agg(F.count("*").alias("games")).orderBy("age_rating"))

# the counts the text quotes, which no single row of the table above carries
display(games.select(
    F.sum((F.col("age_rating") == 0).cast("int")).alias("rated_0"),
    F.sum((F.col("age_rating") > 0).cast("int")).alias("rated_above_0"),
    F.sum((F.col("age_rating") >= 16).cast("int")).alias("rated_16_plus"),
    F.sum(F.col("age_rating").isNull().cast("int")).alias("out_of_range_null"),
    F.count("*").alias("games")))
```

Table **Visualization 1**

Table

Read literally, **only 656 games out of 55 690 (1.2%) carry any age restriction, and 301 are 16+ or over**. That is a fact about the field, not about the catalogue. Steam does not gate on `required_age` at all: the store keys its age screens off the content descriptors a developer declares in the mature content survey, and off regional board ratings such as ESRB and PEGI (Steamworks (https://partner.steamgames.com/doc/store/age_gate)). This file carries neither, and `required_age` is left at 0 by almost everyone.

The community tags are the only proxy available, and Steam's five descriptors split them in two. Only **Adult Only Sexual Content** and **Frequent Nudity or Sexual Content** require the viewer to affirm they are eighteen; **Some Nudity or Sexual Content**, **Frequent Violence or Gore** and **General Mature Content** are disclosures, not barriers. The brief asks which games are *prohibited*, so the two readings are counted separately.

Chart: combo — bars `games`, line `pct_mature`, on `age_group`.

```
# Tags grouped the way Steam's own content descriptors group: the first list maps to the
# two that force an 18+ affirmation, the second adds the three that only disclose.
AGE_GATED = ["Hentai", "NSFW"]
DISCLOSED = AGE_GATED + [
    "Nudity", "Sexual Content",                                  # some nudity or sexual content
    "Gore", "Violent", "Blood",                                  # frequent violence or gore
    "Mature", "Horror", "Survival Horror", "Psychological Horror",  # general mature content
    "Zombies", "Crime", "War", "World War I", "World War II", "Cold War",
    "Dark", "Dark Fantasy", "Dark Humor", "Dark Comedy", "Demons", "Psychological",
    "Gambling", "Assassin", "Villain Protagonist", "Heist"]

def has_tag(tags):
    return F.size(F.array_intersect(F.map_keys("tags"),
                                    F.array(*[F.lit(t) for t in tags]))) > 0

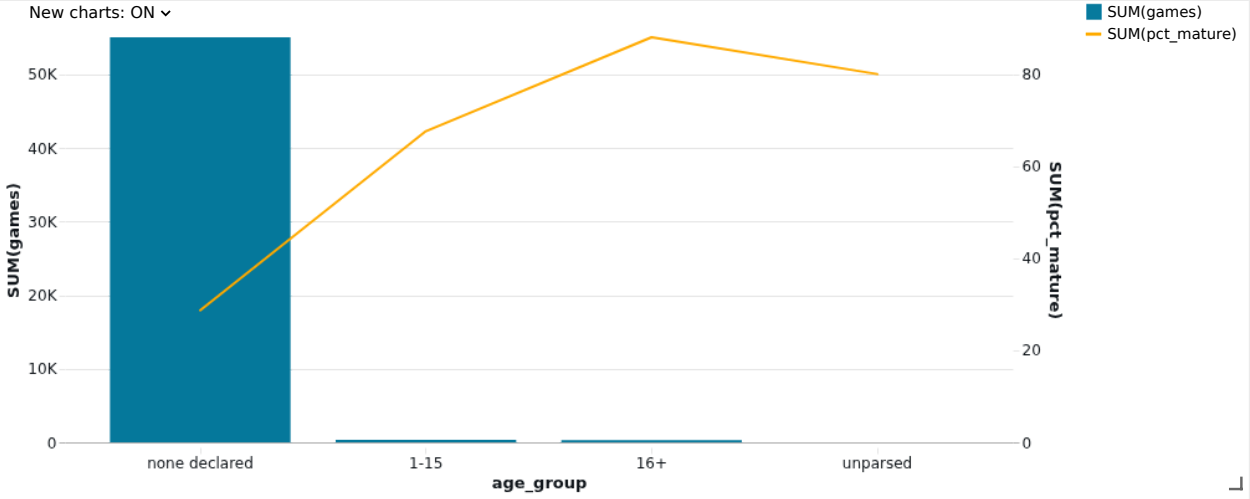
games = (games.withColumn("age_gated", has_tag(AGE_GATED))
              .withColumn("mature", has_tag(DISCLOSED)))

group_rank = (F.when(F.col("age_rating") == 0, 0)
               .when(F.col("age_rating") < 16, 1)
               .when(F.col("age_rating") >= 16, 2)
               .otherwise(3))
age_group = (F.when(group_rank == 0, "none declared")
              .when(group_rank == 1, "1-15")
              .when(group_rank == 2, "16+")
              .otherwise("unparsed"))

display(games.withColumn("rank", group_rank).withColumn("age_group", age_group)
        .groupBy("rank", "age_group").agg(
            F.count("*").alias("games"),
            F.sum(F.col("age_gated").cast("int")).alias("age_gated"),
            F.round(100 * F.avg(F.col("age_gated").cast("int")), 1).alias("pct_gated"),
            F.sum(F.col("mature").cast("int")).alias("any_mature"),
            F.round(100 * F.avg(F.col("mature").cast("int")), 1).alias("pct_mature"))
        .orderBy("rank").drop("rank"))
```

games: pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 47 more fields]

Table **Visualization 1**



4 rows

The field is not noise, it is abandoned. **Where it is filled in it agrees with the tags — 88.0% of the games declaring 16+ carry a mature tag**, against 28.7% of those declaring nothing. But 15 786 games carry one and declare nothing at all.

The strict reading runs the other way. **807 games carry the two tags that map to Steam's age-affirmation descriptors, and 786 of them declare nothing**; only 18 of the 301 declared 16+ games are in that set, 6.0%. Whether Steam's own adult filter already gates them and makes the field redundant, or the publishers who ship them simply fill nothing in, this file cannot say.

The brief's question has three answers, not one: 301 games by the metadata field, 807 by the strictest content reading, 16 295 — 29.3% of the catalogue — by the broadest.

3.6 The best rated games

```
print("games with a 100% positive ratio:", games.filter(F.col("positive_ratio") == 1).count())
display(games.filter(F.col("positive_ratio") == 1)
             .select("name", "positive", "negative", "positive_ratio")
             .orderBy("reviews").limit(5))
```

```
games with a 100% positive ratio: 8634
```

Table

Ranking on the raw ratio returns 8 634 games tied at 100%, most of them on one or two reviews. The Wilson lower bound breaks the tie by asking how much evidence sits behind the score. No review floor is applied below: thin evidence is exactly what the bound already discounts, and adding a threshold on top would only hide the fact that it works.

```
display(games.select("name", "publisher_clean", "reviews",
                     F.round("positive_ratio", 4).alias("positive_ratio"),
                     F.round("wilson_score", 4).alias("wilson_score"),
                     "release_year")
             .orderBy(F.desc("wilson_score")).limit(15))
```

Table

Two of the 8 634 perfect scores survive it — *The Void Rains Upon Her Heart* on 496 reviews and *□□ Inorikaze* on 327 — and the other 8 632 do not. Those two are the largest of the group, which is the bound working rather than leaking.

The top of the list is not made of blockbusters. *Aseprite* is a pixel-art editor, *A Short Hike* and *Patrick's Parabox* are one-person indie games, and the only two titles holding that ratio at scale are **People Playground at 98.9% over 144 569 reviews and Portal 2 at 98.8% over 309 441** — the more useful benchmark, because sustaining the ratio at that volume is the hard part.

Where Ubisoft's own catalogue sits against the market's median ratio, band by band:

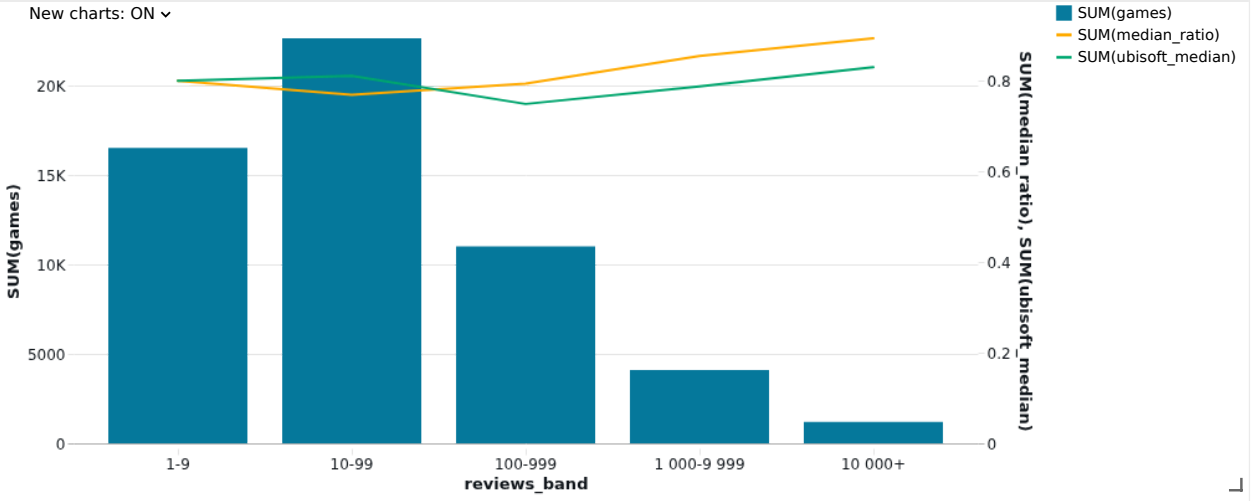
Chart: combo — bars `games`*, lines* `median_ratio` *and* `ubisoft_median`*, on* `reviews_band`*.*

```
ubisoft = F.col("publisher_clean").rlike("(?i)^ubisoft")

review_rank = (F.when(F.col("reviews") < 10, 0).when(F.col("reviews") < 100, 1)
                .when(F.col("reviews") < 1000, 2).when(F.col("reviews") < 10000, 3).otherwise(4))
review_band = (F.when(review_rank == 0, "1-9").when(review_rank == 1, "10-99")
                .when(review_rank == 2, "100-999").when(review_rank == 3, "1 000-9 999")
                .otherwise("10 000+"))

display(games.filter(F.col("reviews") > 0)
    .withColumn("rank", review_rank).withColumn("reviews_band", review_band)
    .groupBy("rank", "reviews_band").agg(
        F.count("*").alias("games"),
        F.round(F.percentile_approx("positive_ratio", 0.5), 3).alias("median_ratio"),
        F.sum(ubisoft.cast("int")).alias("ubisoft_games"),
        F.round(F.percentile_approx(F.when(ubisoft, F.col("positive_ratio")), 0.5), 3)
         .alias("ubisoft_median"))
    .orderBy("rank").drop("rank"))
```

Table **Visualization 1**



5 rows

Reference point — no Ubisoft title appears in the fifteen above, and once the comparison holds review volume constant its catalogue sits below the market in the three bands where it has a real sample: 74.9% against 79.4% between 100 and 999 reviews on 33 titles, 78.8% against 85.5% between 1 000 and 9 999 on 55, 83.0% against 89.4% above 10 000 on 39. It is *above* the market in the 10-99 band, 81.1% against 76.9%, on seven titles — and level with it on the single game it has below ten reviews.

The benchmark for the next game is therefore not the fifteen names above but its own back catalogue, which this table places four to seven points behind the market everywhere it competes in numbers.

4. Genres

`genre` holds several labels per game — three most often, up to sixteen, and none at all for 160 games (2.4a). Exploding it gives one row per (game, genre), so a game counted under Action is also counted under RPG, and the shares below add up to more than 100% by construction.

```
# the raw comma-separated string is dropped: the exploded label replaces it
genre_rows = materialise(games.drop("genre").select("*", F.explode("genres").alias("genre")),
                         "steam_genre_rows")
print(f"{genre_rows.count():,} (game, genre) rows for {games.count():,} games")
```

```
genre_rows:  pyspark.sql.connect.dataframe.DataFrame = [id: string, appid: long ... 47 more fields]
157,110 (game, genre) rows for 55,690 games
```

4.1 What is on the shelf

Chart: bar, `genre` x `games`.

```
display(genre_rows.groupBy("genre").agg(F.count("*").alias("games"))
        .withColumn("pct_of_catalogue", F.round(100 * F.col("games") / games.count(), 1))
        .orderBy(F.desc("games")).limit(15))
```

Table **Visualization 1**

Indie is on 71% of the catalogue — and it is not a genre. Neither are *Early Access* (11%) nor *Free to Play* (6%): they describe how a game is funded and sold, not what it is. Excluding those three, the shelf is **Action (43%), Casual (40%), Adventure (38%), Strategy (20%), Simulation (19%), RPG (17%)**, then Sports (4.8%), Racing (3.9%) and Massively Multiplayer (2.6%) — **nine real genres**, the set the rest of section 4 reads on.

The field mixes more than that. Its 28 labels cover game genres, funding and release states (*Indie*, *Early Access*, *Free to Play*), content warnings (*Violent*, *Gore*, *Nudity*, *Sexual Content*), eleven software categories (*Utilities*, *Photo Editing*, *Design & Illustration*, *Game Development*, …) and one stray *Movie*. The software rows are software: **Wallpaper Engine, Blender, Aseprite, Godot Engine and Source Filmmaker** are all in this catalogue because Steam types them as games, which the `type == 'game'` filter in 2.1 cannot separate. None of those labels reaches 700 games, but they are not removed either, so they appear in the tables below next to real genres.

Section 4 keeps every label in its tables all the same, because the rows that are not genres carry findings of their own.

4.2 Which genres are liked

Every genre carrying at least 100 games, with no review floor. A floor would lift each genre by about two points — better-reviewed games are better rated, as 3.6 shows — and drop seven of the twenty-two, including the lowest-rated one.

Chart: bar, `genre` *x* `median_positive_ratio`.

```
display(genre_rows.groupBy("genre")
        .agg(F.count("*").alias("games"),
             F.round(F.percentile_approx("positive_ratio", 0.5), 3).alias("median_positive_ratio"),
             F.round(F.sum("positive") / (F.sum("positive") + F.sum("negative")), 3).alias("pooled_ratio"))
        .filter(F.col("games") >= 100).orderBy(F.desc("median_positive_ratio")))
```

Table **Visualization 1**

Read on the nine labels that are actually game genres, eight fit inside five and a half points — **Casual 0.803, Adventure 0.797, Action 0.789, RPG 0.779, Strategy 0.769, Racing 0.754, Sports 0.750, Simulation 0.748** — and the ninth is nowhere near them. **Massively Multiplayer sits at 0.648**, ten points below the lowest of the eight and lowest on the pooled column too at 0.731 — the only row under it is *Violent*, a content warning rather than a genre. Live-service games are judged on servers, monetisation and updates long after launch, and this is what that judgement looks like in aggregate.

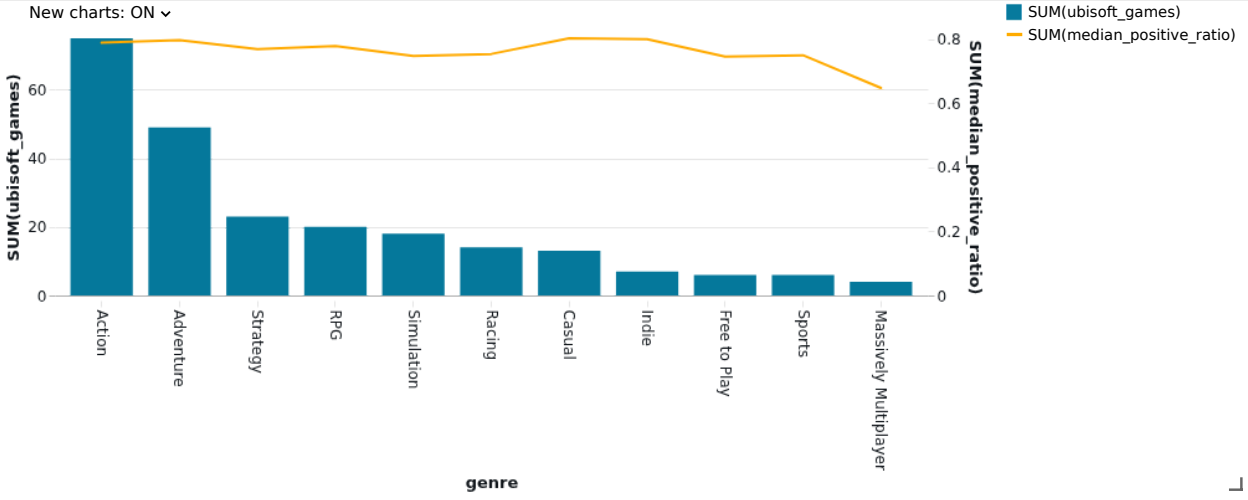
The two columns answer different questions, and the gap between them is the interesting part. The median weights every game equally, so it reads *the typical game on the shelf*. The pooled ratio weights every review equally, so it reads *the average review*. Where the two agree the genre is homogeneous; where they diverge, a handful of titles is doing all the talking. Indie reads 0.800 typical against 0.885 pooled — its hits are much better liked than its median game. Photo Editing takes that to the absurd, 0.750 against 0.977, because a single title — *Wallpaper Engine* — carries almost every review written about those 105 rows: the median describes a shelf of small tools nobody rates highly, the pooled column describes Wallpaper Engine. Which one decides here follows from the question — a studio picking a genre will be one game, not the genre's aggregate, so the median is the column that matters and the pooled one is the check.

Where Ubisoft's own catalogue already sits on that scale:

Chart: combo — bars `ubisoft_games` *, line* `median_positive_ratio` *, on* `genre` *.*

```
display(genre_rows.groupBy("genre").agg(
            F.sum(ubisoft.cast("int")).alias("ubisoft_games"),
            F.count("*").alias("games"),
            F.round(F.percentile_approx("positive_ratio", 0.5), 3).alias("median_positive_ratio"))
        .filter(F.col("ubisoft_games") > 0)
        .orderBy(F.desc("ubisoft_games")))
```

Table **Visualization 1**



11 rows

Ubisoft already publishes where the market is satisfied. Its two largest genres — **Action with 75 titles and Adventure with 49** — are third and second of the nine on the median ratio, 0.789 and 0.797, and it is almost absent from the worst: **4 games in Massively Multiplayer**, the genre ten points below the rest.

One mismatch is worth naming. **Casual leads the nine at 0.803 and Ubisoft has 13 titles in it.** The best-liked genre on Steam is one it barely works in — and 4.1 is the reason rather than a missed opportunity: that shelf is 22 086 hidden-object and puzzle games, a different market.

The table says nothing about how well Ubisoft does *inside* those genres. Its titles carry far more reviews than the median game of the same genre, and 3.6 shows the ratio climbs with review volume, so the comparison that means something is 3.6's — at equal volume, where Ubisoft sits below the market.

Decision — avoid a live-service / MMO structure. The satisfaction penalty is the largest single effect in the genre data, and the only one attached to a design choice rather than to content or to a store category.

4.3 Do publishers have favourite genres

Chart: grouped bar, `publisher_clean` x `pct_of_publisher`, grouped by `genre` — not stacked, since the same games carry several of these labels.

```
top_publishers = [r[0] for r in by_publisher.orderBy(F.desc("games")).limit(8).collect()]

display(genre_rows.filter(F.col("publisher_clean").isin(top_publishers))
        .groupBy("publisher_clean", "genre").agg(F.count("*").alias("games"))
        .withColumn("rank", F.row_number().over(
            Window.partitionBy("publisher_clean").orderBy(F.desc("games"))))
        .filter(F.col("rank") <= 3)
        .join(by_publisher.withColumnRenamed("games", "publisher_games"), "publisher_clean")
        .withColumn("pct_of_publisher",
                    F.round(100 * F.col("games") / F.col("publisher_games"), 1))
        .orderBy("publisher_clean", "rank").drop("rank"))
```

Table **Visualization 1**

Emphatically yes, and the specialisation is near-total. **Of Big Fish Games' 423 titles, 419 are labelled Casual and 393 Adventure** — the same games wearing both labels, not two catalogues side by side, and its third genre stops at 7.

That shape is the answer to the question. A game carries several genres at once, so what a publisher favours is usually a **family of labels rather than a single genre**: Choice of Games is 99.3% RPG, 97.1% Indie and 80.0% Adventure on the same 140 titles — one product described three ways, not three preferences. HH-Games and Sekai Project stack the same way, Casual first with Indie underneath. **8floor is the only pure case in the list**, 100% Casual and then a drop to 10.9%. Square Enix runs a pair, Action at 53.2% with RPG at 50.4%.

Only two of the eight have no formula to point at: **SEGA's top three run 48.5%, 20.0% and 19.4%, Strategy First's 34.4%, 27.8% and 21.9%** — catalogues spread across genres rather than built on one. So six of the eight publish to a formula, and for five of those the formula is a combination.

4.4 Which genres are worth entering

`owners_x_price` averaged over each genre, in millions (2.9 defines it). For one game it is the midpoint of the owners bracket SteamSpy publishes, multiplied by the store's list price — the centre of a range, not a count of copies (2.6). The column averages that over the genre's games. **It is a total accumulated since release — not a price, and not a yearly figure.** Price sits next to it for that reason: what one copy costs, in the same table as what all the copies add up to.

Both are read twice, over the whole genre and over its paid games only. A list price of zero times any number of owners is zero, so every free game enters the first reading as a zero it never earned, and `games` against `paid_games` says how much of each genre that is.

Chart: bar, `genre` x `stock_value_paid_musd`.

```
paid = ~F.col("is_free")

display(genre_rows.groupBy("genre")
        .agg(F.count("*").alias("games"),
             F.sum(paid.cast("int")).alias("paid_games"),
             F.round(F.avg("initial_price_usd"), 2).alias("mean_price_usd"),
             F.round(F.avg(F.when(paid, F.col("initial_price_usd"))), 2).alias("mean_price_paid"),
             F.round(F.avg("owners_x_price") / 1e6, 2).alias("stock_value_musd"),
             F.round(F.avg(F.when(paid, F.col("owners_x_price"))) / 1e6, 2)
              .alias("stock_value_paid_musd"))
        .filter(F.col("games") >= 150)
        .orderBy(F.desc("stock_value_paid_musd")))
```

Table **Visualization 1**

Massively Multiplayer leads at 10.93 M$ a paid game, ahead of RPG at 3.67 and Action at 3.04, while the two largest shelves in the catalogue carry the least: **Indie 0.98 M$ over 39 681 games, Casual 0.43 M$ over 22 086**.

The two readings of a genre differ by exactly how much of it is free, and the identity is visible in the table: `mean_price_usd` is `mean_price_paid` scaled by `paid_games / games`, on every row to within rounding. **The row it wrecks is MMO** — 686 paid games out of 1 460, so the published $5.20 was never a low price, it was a 53% share of free games. On its paid half MMO charges **$11.08, the most of any real genre**, and its stock value *rises* to 10.93 M$ instead of falling: the zeros were holding it down, not propping it up. The eight other real genres run 84.5% to 88.7% paid, so the identity only shaves 11 to 15% off each — enough to compress the published prices, not to rearrange them much. Five of the eight hold their position; only the top three swap, RPG, Sports and Simulation sitting within 21 cents of each other on paid prices. MMO, shaved by 53%, moves from the cheapest real genre to the dearest.

Free to Play is the extreme of it: 149 paid games out of 3 393, $0.30 published against $6.82 paid, 0.03 M$ against 0.69. Free games do not earn nothing — a list price of zero times any number of owners is zero, and the in-game economy behind them is invisible here.

Where the value comes from is then readable directly. **MMO and RPG charge almost the same, $11.08 against $10.97, and MMO carries three times the stock value.** The gap is not what a studio can ask, it is how many people end up owning the game — an audience, and an in-game economy behind it, that a premium single-player release cannot buy. The software labels make the opposite case: **the six highest paid prices in the table, $27.68 to $36.79, are all software, and not one of them reaches 1.4 M$.** That is what a niche tool at a high price looks like.

The most lucrative genres, then: Massively Multiplayer first and by a wide margin — 10.93 M$ a paid game, three times the 3.67 of RPG and the 3.04 of Action, which come next. Nothing else reaches 2.25. At the other end Casual closes the nine real genres at 0.43 M$.

One thing to carry alongside that ranking: 4.2 puts MMO last of the nine on satisfaction, ten points below the eight others. The genre that accumulates the most value is the one whose players like it least — worth knowing before reading this column as a shopping list.

4.5 Is any genre emerging

Chart: grouped bar, `genre` *x* `pct_2017` *and* `pct_2022` *.*

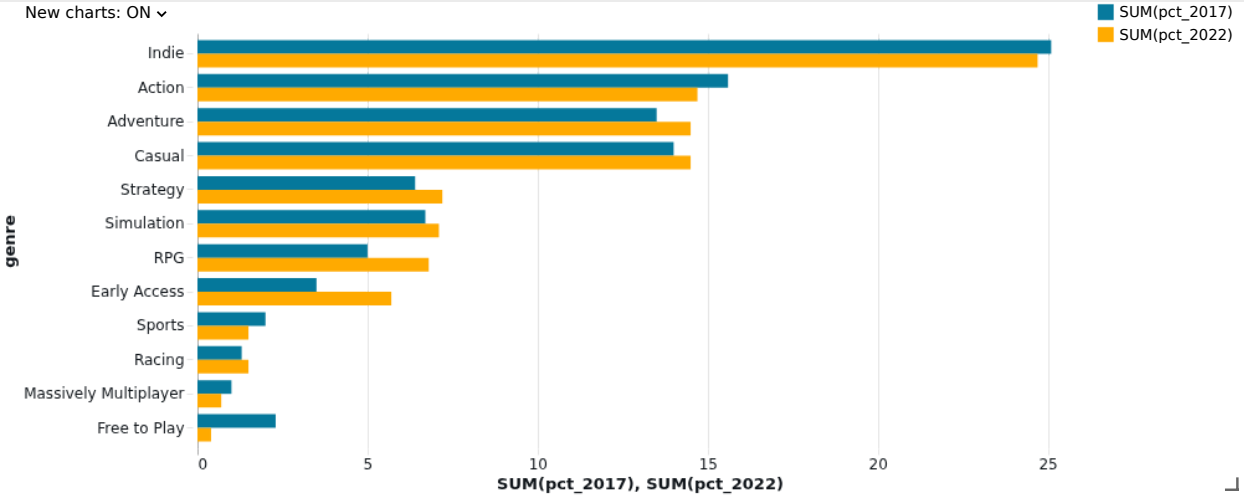
```
mix = (genre_rows.filter(F.col("release_year").isin(2017, 2022))
       .groupBy("release_year", "genre").agg(F.count("*").alias("games")))
year_totals = mix.groupBy("release_year").agg(F.sum("games").alias("total"))

genre_mix = (mix.join(year_totals, "release_year")
    .withColumn("pct", F.round(100 * F.col("games") / F.col("total"), 1))
    .groupBy("genre").pivot("release_year", [2017, 2022]).agg(F.first("pct"))
    .withColumnRenamed("2017", "pct_2017").withColumnRenamed("2022", "pct_2022")
    .withColumn("shift_pts", F.round(F.col("pct_2022") - F.col("pct_2017"), 1)))

display(genre_mix.orderBy(F.desc("pct_2022")).limit(12))
```

```
genre_mix:  pyspark.sql.connect.dataframe.DataFrame = [genre: string, pct_2017: double ... 2 more fields]
mix:  pyspark.sql.connect.dataframe.DataFrame = [release_year: integer, genre: string ... 1 more field]
year_totals:  pyspark.sql.connect.dataframe.DataFrame = [release_year: integer, total: long]
```

Table **Visualization 1**



12 rows

Five years apart, the mix barely moves: Indie 25.1% → 24.7%, Action 15.6% → 14.7%, Casual 14.0% → 14.5%. The only shifts worth naming are **Early Access, 3.5% → 5.7%**, and **Free to Play collapsing from 2.3% to 0.4%** — the latter partly a labelling change, since free games kept their share of the catalogue while the genre tag stopped being applied.

No genre is emerging. A concept does not need to catch a wave here, because there isn't one; it needs to be good in a category that already pays.

5. Platforms

5.1 What Steam runs on

```
display(games.select(
    F.count("*").alias("games"),
    F.sum(F.col("windows").cast("int")).alias("windows"),
    F.sum(F.col("mac").cast("int")).alias("mac"),
    F.sum(F.col("linux").cast("int")).alias("linux")))

display(games.groupBy("windows", "mac", "linux").agg(F.count("*").alias("games"))
        .orderBy(F.desc("games")))
```

Table

Table

	windows	mac	linux	games
1	true	false	false	41271
2	true	true	true	6806
3	true	true	false	5951
4	true	false	true	1647
5	false	true	false	11
6	false	false	true	3
7	false	true	true	1

7 rows

Yes for Windows, no for the other two. 55 675 of the 55 690 games run on Windows — 99.97%, and the 15 that do not are curiosities. Mac reaches 12 769 games and Linux 8 457, 22.9% and 15.2% of the catalogue. **The largest group on the store is Windows-only: 41 271 games, three quarters of it.**

Where Ubisoft's own catalogue sits against that, read the same way:

```
display(games.filter(ubisoft).select(
    F.count("*").alias("games"),
    F.sum(F.col("windows").cast("int")).alias("windows"),
    F.sum(F.col("mac").cast("int")).alias("mac"),
    F.sum(F.col("linux").cast("int")).alias("linux")))

display(games.filter(ubisoft).groupBy("windows", "mac", "linux")
        .agg(F.count("*").alias("games")).orderBy(F.desc("games")))
```

Table

Table

Ubisoft ports less than the store does, by a wide margin. All 135 of its games run on Windows, 6 reach Mac and 2 reach Linux — 4.4% and 1.5%, against the catalogue's 22.9% and 15.2%. **128 of the 135, 94.8%, are Windows-only**, where the store as a whole is at 74.1%.

The total reads 135 here and 128 in 3.1 because this filter takes every normalised publisher string beginning with *Ubisoft* — the three that 2.5 leaves — while 3.1's ranking counts one spelling at a time.

5.2 Do certain genres get ported more

Chart: bar, `genre` *x* `pct_mac` *and* `pct_linux`*.*

```
display(genre_rows.groupBy("genre").agg(
            F.count("*").alias("games"),
            F.round(100 * F.avg(F.col("mac").cast("int")), 1).alias("pct_mac"),
            F.round(100 * F.avg(F.col("linux").cast("int")), 1).alias("pct_linux"))
        .filter(F.col("games") >= 150).orderBy(F.desc("pct_mac")))
```

Table **Visualization 1**

Yes, but the effect is small. Among the nine real genres the Mac share runs from **Strategy at 27.6% down to Massively Multiplayer at 18.5%**, with Action near the bottom at 19.2% — a nine-point spread, a ratio of 1.5, inside a band that never exceeds 28%. Linux orders them almost the same way, Strategy top again at 16.8% and Sports last at 10.8%. Only one label beats Strategy on Mac, and it is not a genre: *Game Development* at 32.7%, software rather than a game (4.1).

So no genre is a Mac genre or a Linux genre. The gap between the most and the least ported real genre is smaller than the gap between any of them and Windows' 99.97%.

6. What the study says about the next game

Seven readings, each from the section that produced it. The left column is measured; the right is what it implies for a product brief — and the gap between the two matters, because nothing here is a demonstrated payoff.

	what the analysis shows	what it means for the brief
Competition	releases went from 2 575 in 2015 to 8 823 in 2021, and the second half of the year is systematically the crowded one — October 4 451 against January 3 096 (3.2)	the shelf is three times as full as it was, so standing out matters more than timing; a first-half launch meets the six lightest months of the year
Price	the store is cheap — 78.6% of the catalogue is free or under $10, 42.2% under $5 alone, and 95.6% of paid games end on `.99` (3.3)	a premium price puts the game outside where four fifths of the catalogue sits. This study measures that it is unusual, not that it pays
Languages	English is on 99% of games; the tier below runs German, French, Russian, Simplified Chinese, Spanish, Japanese, Italian at 25.2% down to 16.7%, then steps to 12.1%. More than half the catalogue ships in a single language (3.4)	English plus those seven, eight in all, cut where the catalogue's own list steps down. How far past eight to go is not decidable here
Age rating	`required_age` is abandoned — 301 games declare 16+ — while 29.3% of the catalogue carries mature content tags and 807 carry the two behind Steam's 18+ gate (3.5)	mature content is the norm rather than an edge case, and the metadata field is no guide. Steam gates on developer-declared descriptors, so this is a disclosure question, not a market one
Genre, satisfaction	Ubisoft's two largest genres — Action with 75 titles, Adventure with 49 — are third and second of the nine on the median positive ratio, and it has 4 games in Massively Multiplayer, ten points below the rest (4.2)	the existing catalogue already sits in the well-liked genres. Nothing in the data argues for leaving them
Genre, value	the same genres carry the value: RPG 3.67 M$ a paid game, Action 3.04, Strategy 2.23, Adventure 2.10, against Casual's 0.43 (4.4)	Ubisoft's shelf and the lucrative shelf are the same shelf. MMO leads both columns and fails on satisfaction, which is the one genre the two readings disagree about
Platforms	Windows is 99.97% of the store, Mac 22.9%, Linux 15.2%. Ubisoft ships 4.4% Mac and 1.5% Linux, and 94.8% of its games are Windows-only against the store's 74.1% (5.1)	Ubisoft ports five to ten times less than the market. Whether closing that gap pays is measured nowhere in this notebook

The quality bar is 3.6's. Above 10 000 reviews the median positive ratio on Steam is **89.4%**, and Ubisoft's own 39 titles in that band sit at **83.0%** — so a release of that size needs to beat its own catalogue by six points just to be an ordinary game of its class.

7. What this dataset cannot decide

It stops on 11 November 2022. Every 2022 figure covers ten and a half months, and nothing after that date exists. The Steam Deck shipped in February 2022, so it is eight months old in this snapshot — whatever it changed about Linux ports is not in here yet.

There are no sales. `owners` is SteamSpy's *estimate*, published as a bracket, and 68% of the catalogue falls in the bottom one. `owners_x_price` multiplies that bracket's midpoint by the list price, so what it counts is what a customer would pay, not what a publisher receives: the store's commission, regional pricing, discounts, refunds, bundles and free keys are all outside it. And it values every free game at zero, which is why section 4.4 reads *Free to Play: 0.03 M$ a game* for a business model that funds some of the largest games in the table.

Reviews are not players. They stand in for reach throughout, and how tightly the two track is not measured here — enough to rank on, not enough to size with.

Publishers are strings, not firms. `publisher` is free text. 2.5 trims whitespace and strips trademark signs, which is why *Ubisoft* falls from five raw spellings to three, but it does not merge *Ubisoft* with *Ubisoft Entertainment* — deciding that two company names are one firm is a judgement, not a cleaning rule. So 3.1 ranks names rather than companies, 4.3's formulas are formulas of names, and the same publisher can count 128 games in one section and 135 in another depending on how the filter is written.

Nothing here is causal. A genre is chosen by studios that already expect a game to sell, so where 4.2 finds a genre better liked and 4.4 finds it worth more, the genre and the games that picked it cannot be separated. Section 6 is a reading of where successful games are, not a recipe for becoming one.

Delisted games are absent. The catalogue is what was on sale in November 2022, so failures that were pulled never appear — every share measured here is, if anything, optimistic.

One store, one region. Steam is not consoles, not mobile, not the Epic store, and the prices are US dollars.

;